@qa_insights

AWS

COMPLETE NOTES

(AWS)

BEGINNER TO ADVANCED

Compute	Storage	Database	Security	Monitoring	Networking
EC2, Lambda	S3, EBS	RDS, DynamoDB	IAM, VPC	CloudWatch	VPC, Route 53

SWIPE TO NEXT →

FOLLOW | SHARE | COMMENT | LIKE

AWS

Amazon Web Services

World's most popular cloud platform by Amazon.
Build, deploy and scale applications on the cloud.

1 What is Cloud Computing?

- Delivery of IT services over the internet.
- On-demand resources, pay-as-you-go.
- No need to manage physical hardware.

2 What is AWS?

- AWS (Amazon Web Services) is a comprehensive cloud platform.
- Offers 200+ fully featured services.
- Global, reliable, secure and scalable.

3 Why Companies Use AWS?

- Reduce infrastructure cost
- Rapid innovation and agility
- Scalable and flexible resources
- High availability and reliability
- Global reach in minutes

4 Benefits of AWS

- Pay-as-you-go pricing
- Elasticity & Scalability
- High Availability
- Security & Compliance
- Wide range of services

REMEMBER

- You don't manage infrastructure.
- You focus on building the business.
- AWS handles the heavy lifting.

AWS Global Infrastructure

- **Regions** – Separate geographic areas.
- **Availability Zones (AZ)** – Multiple data centers in each region.
- **Edge Locations** – CDN for fast content delivery.
- **Local Zones & Wavelength Zones** – Closer to users for low latency.
- **Multiple Data Centers** – Powering millions of applications worldwide.

Top AWS Services

PRO TIP

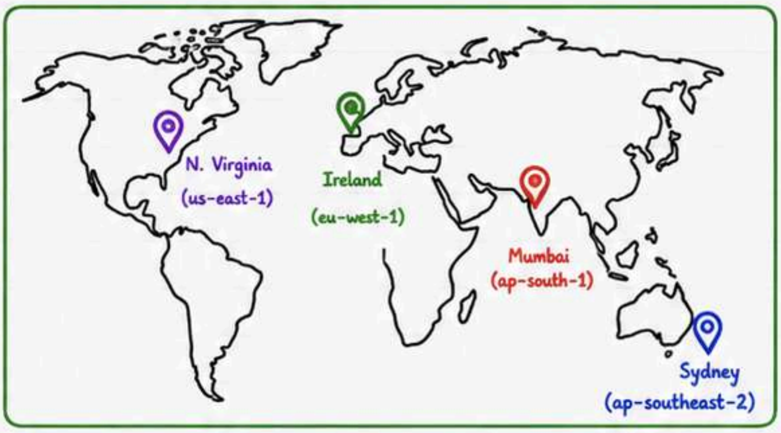
- Start Small, Scale as You Grow.
- Use Managed Services whenever possible.
- Monitor cost and optimize resources.

AWS Global Infrastructure

AWS offers a secure, reliable & global network of data centers to run your applications anywhere in the world.

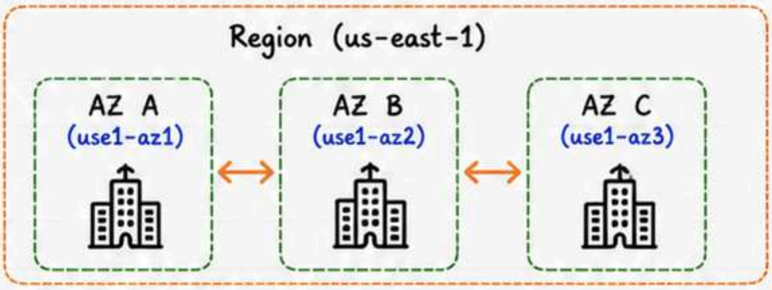
1 Regions

- A Region is a separate geographic area.
- Completely isolated from other Regions.
- Each Region has multiple AZs.
- Example: us-east-1 (N. Virginia)



2 Availability Zones (AZ)

- AZs are one or more data centers with independent power, cooling and networking.
- Designed for high availability.
- AZs are connected with low latency.

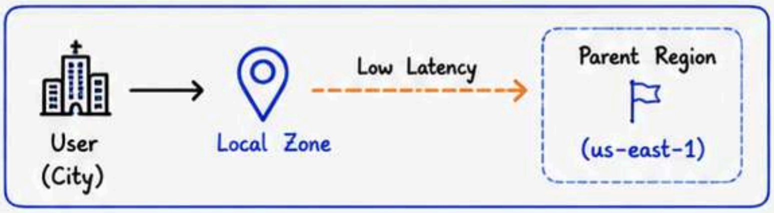


3 Edge Locations

- Used by CloudFront (CDN).
- Deliver content closer to users.
- Not used to run AWS resources.
- Hundreds of edge locations worldwide.

4 Local Zones

- Extend AWS infrastructure closer to large population centers.
- Lower latency for real-time apps.
- Dependent on a parent Region.



REMEMBER

- Region → Multiple AZs
- AZ → Multiple Data Centers
- Edge & Local Zones → Closer to Users

REAL-WORLD EXAMPLE

A user in India accesses your app.
Request goes to nearest Edge/Local Zone
Data is processed in the AWS Region (Mumbai).

@qa_insights

3

IAM (Identity & Access Management)

IAM

IAM helps you securely manage access to AWS services and resources for your users and applications.

① Users

- An individual identity.
- Has unique credentials.
- Can belong to one or more groups.

② Groups

- Collection of users.
- Permissions applied to the group.
- Easier management of access.

③ Roles

- Used by AWS services or external users.
- Temporary access.
- No long-term credentials.

④ Policies

- JSON documents.
- Define permissions (Allow / Deny).
- Attach to Users, Groups or Roles.

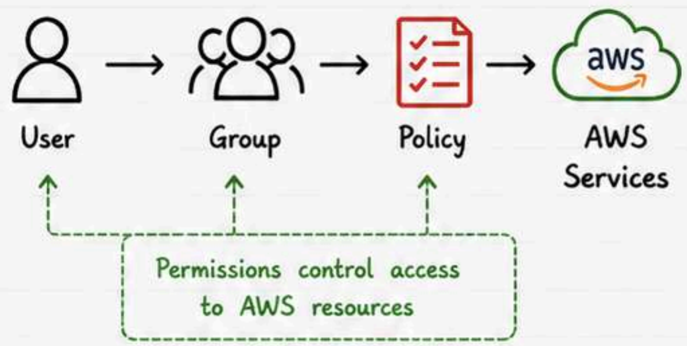
⑤ MFA (Multi-Factor Authentication)

- Extra layer of security.
- Requires a second verification (OTP or Authenticator app).
- Protects against unauthorized access.

⑥ Least Privilege Principle

- Grant minimum permissions necessary.
- Reduces security risks.
- Best practice for all AWS accounts.

How IAM Works



Example Policy (Read Only S3 Access)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [ "s3:ListBucket", "s3:GetObject" ],
      "Resource": [
        "arn:aws:s3:::my-bucket",
        "arn:aws:s3:::my-bucket/*"
      ]
    }
  ]
}
```

REMEMBER

- Users are people.
- Roles are for applications or services.
- Policies define what is allowed.

BEST PRACTICE

- Use Groups to manage permissions.
- Enable MFA for all users.
- Follow Least Privilege always.

EC2 (Virtual Servers)

EC2

Amazon EC2 provides secure, resizable compute capacity in the cloud. You get virtual servers in minutes.

1 What is EC2?

- EC2 = Elastic Compute Cloud
- Virtual servers in AWS cloud.
- Full control of OS, storage, networking & security.

2 Instance Types

- Different types for different workloads.
- Example families: t, m, c, r, i, g
- Choose based on CPU, Memory, Storage, Network.

3 AMI (Amazon Machine Image)

- Pre-configured template.
- Includes OS, software & settings.
- Use AWS Marketplace, or custom AMI.

4 Key Pair

- Used to securely connect to EC2.
- Consists of Public key (.pem) & Private key.
- Keep private key safe!

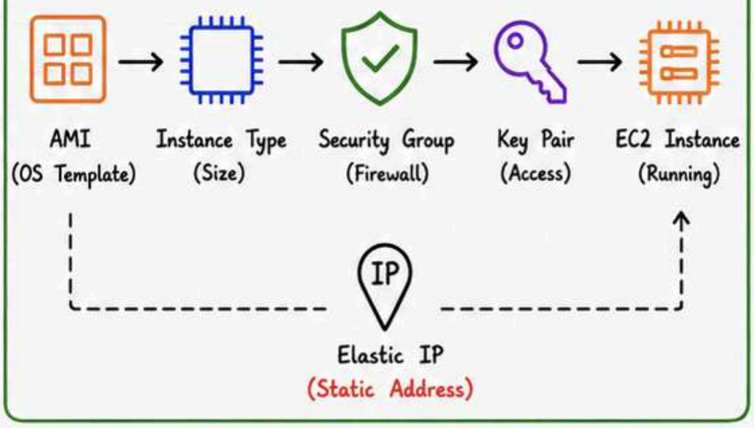
5 Security Groups

- Acts as a virtual firewall.
- Control inbound & outbound traffic.
- Rules based on IP, Port, Protocol.

6 Elastic IP

- Static IPv4 address.
- Not attached to instance by default.
- Re-attach to running instance.

EC2 Launch Flow



Example: Launch EC2 Instance

- [x] 1. Choose an AMI (e.g., Amazon Linux 2)
- [x] 2. Select Instance Type (e.g., t3.micro)
- [x] 3. Configure Security Group (Allow SSH 22)
- [x] 4. Create or Select Key Pair (.pem file)
- [x] 5. Launch Instance
- [x] 6. (Optional) Allocate & Associate Elastic IP

Connect to EC2 (Linux)

```
ssh -i your-key.pem ec2-user@<Public-IP>
```

REMEMBER

- You pay for running EC2 instances.
- Stop = Stop charging (EBS data safe).
- Terminate = Delete (EBS deleted if not attached).

BEST PRACTICE

- Use Least Privilege in Security Groups.
- Use IAM Roles for EC2 (not access keys).
- Enable monitoring with CloudWatch.
- Regularly stop unused instances.

Storage Services

AWS provides secure, durable and scalable storage solutions for every type of data.

1 S3

- Object storage
- Highly scalable
- 99.999999999% durability
- For any type of data

2 EBS

- Block storage
- For EC2 instances
- High performance
- Data tied to AZ

3 EFS

- File storage
- Shared across multiple instances
- Auto scaling
- Used with Linux

4 Glacier

- Archive storage
- Very low cost
- Data retrieval takes minutes-hours
- For backup & archive

Storage Comparison

Service	Type	Access	Durability	Performance	Use Case	Cost
S3	Object	Internet (HTTP/HTTPS)	11 9's (99.999999999%)	High	Static websites, backups, media, data lakes	$
EBS	Block	EC2 (Attached)	99.999%	High	OS, databases, applications requiring low latency	$$
EFS	File	NFS (Shared)	99.999%	Medium	Shared files, content management	$$
Glacier	Archive	Internet (Retrieval)	11 9's (99.999999999%)	Low	Long-term backup, compliance, archives	$

When to Use Which?

S3

Use for objects, backups, logs, static content, data lakes.

EBS

Use for EC2 boot volumes, databases, high performance apps.

EFS

Use for shared file systems across multiple EC2 instances.

Glacier

Use for long-term archives, compliance, cold data storage.

REMEMBER

- S3 = Object Storage (Internet scale)
- EBS = Block Storage (EC2)
- EFS = File Storage (Shared)
- Glacier = Archive Storage (Low cost)

BEST PRACTICE

- Choose the right storage for the right workload.
- Use Lifecycle policies in S3 to reduce costs.
- Encrypt data at rest and in transit.

Networking Basics

VPC

AWS Networking helps your resources communicate securely with the internet and each other.

1 VPC (Virtual Private Cloud)

- Your isolated network in AWS.
- Control IP ranges, subnets, route tables, gateways.
- Resources in a VPC can communicate privately.

2 Public vs Private Subnet

Public Subnet

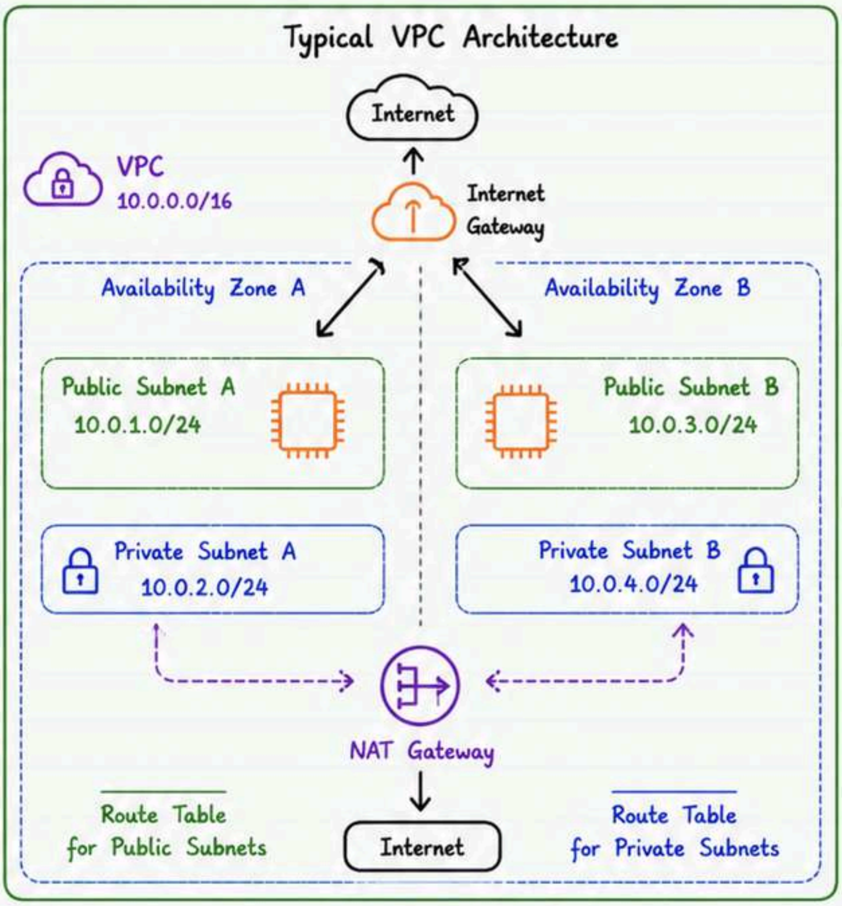
- Has route to Internet Gateway
- Resources are publicly accessible

Private Subnet

- No direct route to Internet
- More secure and isolated

3 Internet Gateway (IGW)

- Connects your VPC to the internet.
- Enables internet access for resources in public subnets.



4 Route Table

- Contains rules (routes) to direct traffic.
- Associated with a subnet.
- Decides where network traffic goes.

5 NAT Gateway

- Allows resources in private subnets to access the internet.
- Blocks inbound traffic from internet.
- One NAT Gateway per AZ for high availability.

6 CIDR Basics

192.168.0.0/16

- CIDR defines IP address range in a network.
- Format: A.B.C.D/n
- Example: 10.0.0.0/16 (65536 IPs)
- /24 = 256 IPs, /16 = 65536 IPs

REMEMBER

- Public Subnet → Internet access
- Private Subnet → No direct Internet
- NAT Gateway → Outbound Internet for Private Subnet

BEST PRACTICE

- Use Private Subnets for databases and backend services.
- Place NAT Gateway in each AZ.
- Use Route Tables wisely to control traffic flow.

Load Balancer & Auto Scaling

AWS

Distribute traffic, handle load & scale your applications automatically for high availability.

1 Why Load Balancer?

- Distribute incoming traffic across multiple instances.
- Prevent overload on a single server.
- Improve availability and fault tolerance.
- Perform health checks.

2 ALB (Application Load Balancer)

- Operates at Layer 7 (HTTP/HTTPS).
- Routes based on URL / Host / Path / Headers.
- Ideal for web applications and microservices.
- Supports SSL termination.

3 NLB (Network Load Balancer)

- Operates at Layer 4 (TCP/UDP).
- Handles millions of requests per second.
- Ultra low latency.
- Ideal for high performance applications.

4 Auto Scaling Group (ASG)

- Automatically adjusts the number of instances.
- Maintains application availability.
- Replace unhealthy instances automatically.
- Works with Load Balancers.

5 Scaling Policies

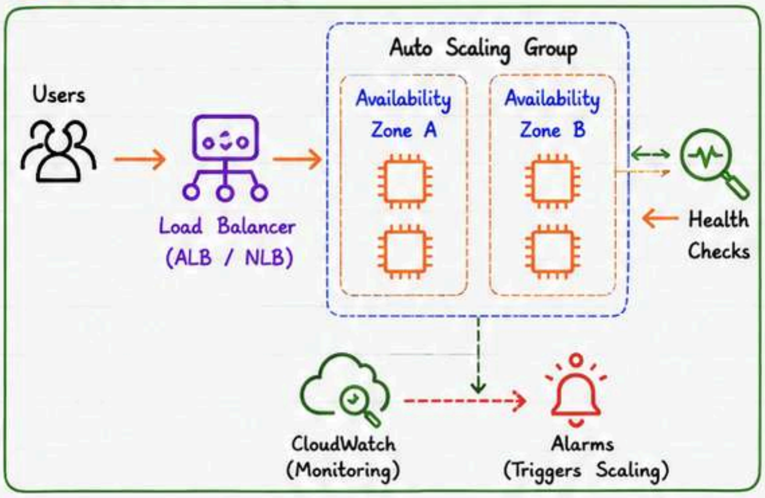
Target Tracking (Recommended)

- Maintain a specific metric (e.g., CPU = 50%).
- Automatically adjusts capacity.

Step Scaling

- Scale based on CloudWatch alarms.
- Define steps for scale out or scale in.

6 High Availability



ALB vs NLB

Feature	ALB (Application)	NLB (Network)
Layer	Layer 7 (HTTP/HTTPS)	Layer 4 (TCP/UDP)
Use Case	Web apps, APIs, Microservices	High performance apps, Gaming, IoT
Routing	Path, Host, Query, Headers	IP, Port
Performance	Good	Ultra High
Static IP	No	Yes
SSL Termination	Yes	No (Pass-through)

REMEMBER

- ALB = Smart routing for applications.
- NLB = Speed & performance at network level.
- ASG = Scale out/in automatically.

BEST PRACTICE

- Use ALB for most web applications.
- Use NLB for high throughput & low latency workloads.
- Always enable Health Checks.
- Spread instances across multiple AZs.

Route 53 & DNS

Amazon Route 53 is a scalable and highly available DNS web service.

Route 53

1 DNS Basics

DNS

- DNS translates domain names to IP addresses.
- Works like the internet phonebook.
- Uses a distributed, hierarchical system.

2 Hosted Zone

.com

- A container for DNS records.
- Public Hosted Zone (internet facing).
- Private Hosted Zone (inside VPC only).

3 Record Types

A

- A – Maps domain to IPv4.
- AAAA – Maps domain to IPv6.
- CNAME – Alias of another domain.
- MX – Mail exchange record.
- TXT – Text information.

4 Health Checks

- Monitor the health of your resources.
- Checks via HTTP, HTTPS, TCP, or DNS.
- Used for failover routing.

5 Routing Policies

- Simple – Default, single resource.
- Weighted – Split traffic by weight.
- Latency – Route based on latency.
- Failover – Active-passive setup.
- Geolocation – Route by user location.
- Geoproximity – Route close to users.

6 Domain Flow

User Enters www.example.com → 1. Browser checks cache → 2. Query to Route 53 DNS → 3. Route 53 resolves IP → 4. Connection to the server → 5. Response to the user

www.

53

Data is returned to the user

REMEMBER

- DNS = Domain Name System
- Route 53 = DNS service by AWS
- Converts domain names to IP addresses

BEST PRACTICE

- Use appropriate routing policy for your use case.
- Enable Health Checks for critical applications.
- Use Alias Records to route to AWS resources.

@qa_insights

Databases

AWS

AWS offers a wide range of database services for every application need.

1 RDS

- Managed relational database service.
- Supports MySQL, PostgreSQL, MariaDB, Oracle, SQL Server.
- Easy setup, backups, patching & scaling.

2 Aurora

- AWS native relational DB compatible with MySQL & PostgreSQL.
- High performance & availability.
- Auto scaling storage up to 128 TB.

3 DynamoDB

- Fully managed NoSQL key-value database.
- Single-digit millisecond latency.
- Auto scaling with seamless performance.

4 ElastiCache

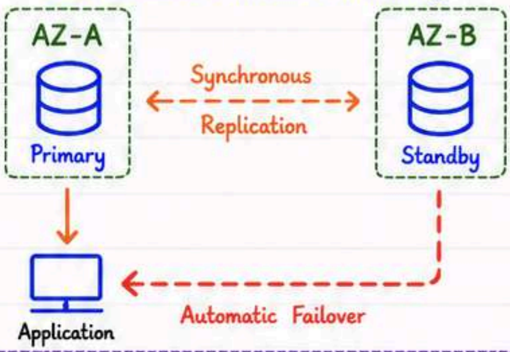
- In-memory caching service.
- Supports Redis and Memcached.
- Improves application performance.

5 SQL vs NoSQL

Feature	SQL (RDS / Aurora)	NoSQL (DynamoDB)
Data Model	Table (Rows & Columns)	Key-Value / Document
Schema	Fixed Schema	Schema-less
Scaling	Vertical Scaling	Horizontal Scaling
Joins	Supported	Not Supported
ACID	Full ACID	Eventually Consistent
Use Cases	Transactions, ERP, CRM, Reporting	High scale apps, IoT, Gaming, User Profiles

6 Multi-AZ (High Availability)

Multi-AZ provides automatic failover and increases database availability.



When to Use What?

RDS
Use for traditional relational databases with mature applications.

Aurora
Use for high performance relational apps needing scalability & high availability.

DynamoDB
Use for NoSQL apps with massive scale and low latency requirements.

ElastiCache
Use to cache data and reduce load on databases.

REMEMBER

- RDS & Aurora = Relational (SQL)
- DynamoDB = NoSQL Key-Value
- ElastiCache = In-Memory Cache
- Multi-AZ = High Availability

BEST PRACTICE

- Enable Multi-AZ for critical databases.
- Choose the right DB for the right workload.
- Use ElastiCache to improve read performance.
- Monitor DB performance and optimize regularly.

Serverless Computing

AWS

Build and run applications without managing servers. Just write code and focus on logic!

① Lambda

- Compute service that runs your code in response to events.
- Supports multiple languages (Python, Node.js, Java, etc.)
- Pay only for the code execution time.

② API Gateway

- Fully managed service to create, publish, maintain and secure APIs.
- Acts as a front door for your backend (Lambda or other services).
- Supports REST and WebSocket APIs.

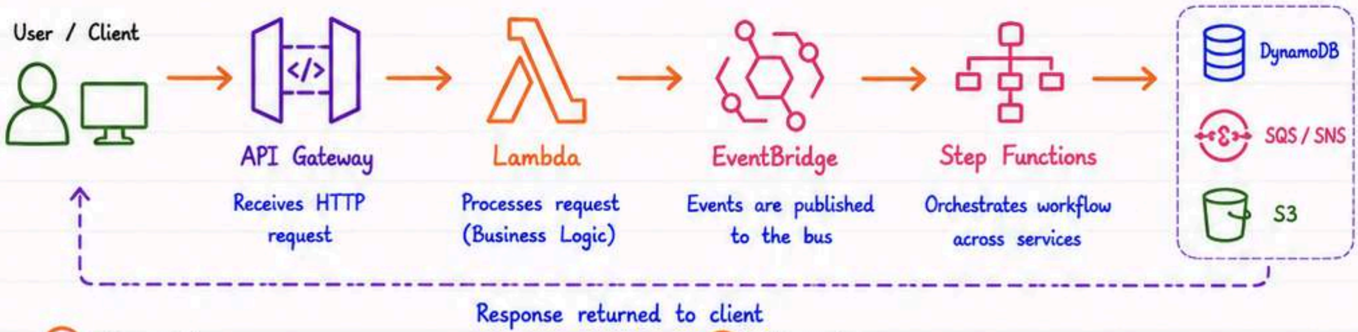
③ EventBridge

- Event bus that receives events from AWS services, SaaS apps, or custom apps.
- Routes events to targets like Lambda, SQS, SNS, Step Functions, etc.
- Enables event-driven architectures.

④ Step Functions

- Orchestrates complex workflows using state machines.
- Coordinates Lambda functions and other AWS services.
- Handles retries, errors and parallel execution.

Typical Serverless Application Flow



⑤ Benefits

- ✓ No server management – AWS handles infrastructure.
- ✓ Pay-per-use – You pay only for what you use.
- ✓ Auto scaling – Scales automatically with demand.
- ✓ High availability – Built-in fault tolerance.
- ✓ Faster development – Focus on code, not operations.

⑥ Use Cases

- Web & mobile backends
- File processing & data transformation
- Real-time data processing & streaming
- Scheduled tasks & automation
- IoT data ingestion & analytics

REMEMBER

- Serverless does NOT mean no servers. You don't manage them, AWS does! ☺
- Lambda has execution timeout (max 15 min).
- Keep functions small and single purpose.

BEST PRACTICE

- Design for events, not requests.
- Use least privilege IAM roles.
- Monitor with CloudWatch (logs, metrics, alarms).

Containers on AWS

AWS

Run, manage and scale containerized applications using AWS container services.

① ECS
(Elastic Container Service)

- Fully managed container orchestration service.
- Uses AWS native services (IAM, ALB, CloudWatch, etc.)
- Supports EC2 launch type and Fargate.

② EKS
(Elastic Kubernetes Service)

- Managed Kubernetes service on AWS.
- Full Kubernetes compatibility.
- Ideal for complex, enterprise-grade Kubernetes workloads.

③ Fargate
(Serverless Containers)

- Serverless compute engine for containers.
- No need to manage servers or clusters.
- Pay only for vCPU and memory used.

④ Docker Integration

- Build container images with Docker.
- Push to Amazon ECR (Elastic Container Registry).
- Deploy to ECS/EKS or run on Fargate.

⑤ Kubernetes Support

- EKS provides fully managed Kubernetes control plane.
- Supports add-ons, auto-scaling, upgrades and observability.
- 100% Kubernetes API compatibility.

Typical Container Deployment Flow on AWS

Code → Build Image → Push to ECR → Deploy → Run Containers → Monitor & Scale

Code	Build Image	Push to ECR	Deploy	Run Containers	Monitor & Scale
</>	Docker	Amazon ECR	ECS / EKS / Fargate	Your Application	CloudWatch / Auto Scaling

When to Choose Which?

Service	Best For	Key Advantage
ECS (EC2)	Teams who want simplicity with AWS integration and more control over infrastructure.	Tight AWS integration, easy to use, supports EC2 & Fargate.
EKS	Teams needing full Kubernetes power and portability across environments.	Full Kubernetes compatibility, rich ecosystem, open source.
Fargate	Teams who want serverless experience without managing servers.	No servers to manage, auto scaling, pay per use.

REMEMBER

- ECS = Simplicity + AWS Native.
- EKS = Kubernetes Power + Flexibility.
- Fargate = Serverless Containers.

BEST PRACTICE

- Store images in ECR, not Docker Hub.
- Use IAM roles for tasks/pods (no hardcoded keys).
- Enable logging and monitoring (CloudWatch).

@qa_insights

12

CI/CD on AWS

DEV OPS AWS

Automate build, test and deploy your applications using AWS CI/CD services.

1 CodeCommit

- Fully managed source control service.
- Secure Git repositories hosted in AWS.
- Supports code versioning and collaboration.
- Integrated with IAM for security.

2 CodeBuild

- Fully managed build service.
- Compiles source code, runs tests.
- Supports multiple languages and build environments.
- Produces build artifacts.

3 CodeDeploy

- Automates application deployments.
- Supports EC2, Lambda and On-premises.
- Blue/Green and In-place deployments.
- Reduces downtime and deployment failures.

4 CodePipeline

- Orchestrates CI/CD pipeline.
- Automates release process end to end.
- Integrates with AWS tools and third-party tools.
- Visualize and manage deployment workflows.

Deployment Flow

Developer Push Code → CodeCommit: Store source code in Git repository → CodeBuild: Build source code & run tests → CodeDeploy: Deploy application to target → EC2 / Lambda / On-premises

Feedback / Logs

DevOps Pipeline (End to End)

1. Source → 2. Build → 3. Test → 4. Deploy → 5. Monitor

CodeCommit
Developers commit code to repository

CodeBuild
Compile code, run tests, generate artifacts

Automated Tests
Unit, Integration, Security & Quality checks

CodeDeploy
Deploy to staging / production safely

CloudWatch
Monitor logs, metrics and alarms

REMEMBER

- CodePipeline orchestrates the entire workflow.
- Each service has its own IAM permissions.
- Automate everything to reduce human errors.

BEST PRACTICE

- Keep pipeline stages small and focused.
- Use automatic rollback (CodeDeploy Blue/Green).
- Monitor pipelines and set up alerts in CloudWatch.

Monitoring & Logging

AWS

Monitor performance, track activity and collect logs to keep your AWS resources healthy and secure.

1 CloudWatch

- Monitor AWS resources and applications.
- Collect metrics, logs and events.
- Visualize with dashboards.
- Set alarms and take automated actions.

2 CloudTrail

- Track API calls and user activity.
- Provides audit logs of all actions.
- Helps with security analysis and compliance.

3 AWS Config

- Tracks resource configurations.
- Records configuration changes over time.
- Evaluate compliance against rules.
- Helps in governance and auditing.

4 X-Ray

- Analyze and debug distributed applications.
- End-to-end request tracing.
- Identify performance bottlenecks.

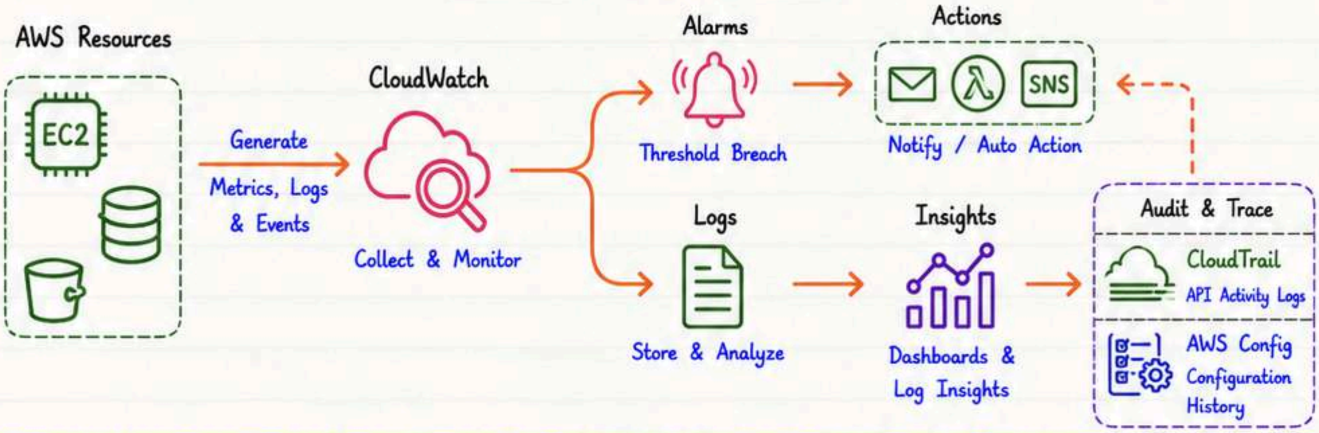
5 Alarms

- Watch metrics and trigger alerts.
- Send notifications via SNS, Email, Lambda.
- Take automated actions on threshold breach.

6 Logs

- Centralized log collection.
- Store, search and analyze logs.
- Supports CloudWatch Logs Insights.
- Retention and archiving support.

Monitoring & Logging Flow



REMEMBER

- Use CloudWatch for metrics, logs and alarms.
- CloudTrail captures "Who did What and When".
- AWS Config ensures compliance and governance.
- X-Ray helps you debug performance issues.

BEST PRACTICE

- Create meaningful dashboards and alarms.
- Set log retention policies to control costs.
- Enable CloudTrail in all regions.
- Review and act on alarms promptly.

PRO TIP

Combine CloudWatch Logs Insights with X-Ray traces for faster troubleshooting!

USE CASES

- Detect issues early and reduce downtime.
- Meet security and compliance requirements.
- Optimize performance and costs.

Security Services

AWS

Secure your AWS environment, protect data, and detect threats in real-time.

① KMS

- Manage encryption keys used across AWS services.
- Encrypt data at rest in S3, EBS, RDS, etc.
- Customer managed or AWS managed keys.

② Secrets Manager

- Securely store and manage secrets like DB passwords, API keys, tokens.
- Automatic rotation of secrets.
- Integrates with many AWS services.

③ WAF (Web ACL)

- Protects web applications from common threats.
- Filter malicious traffic using rules.
- Integrates with ALB, CloudFront, API Gateway.

④ Shield

- DDoS protection service.
- Shield Standard – Free, automatic protection.
- Shield Advanced – Advanced protection with 24/7 support.

⑤ GuardDuty

- Intelligent threat detection service.
- Monitors for malicious activity and anomalies.
- Uses ML, threat intelligence and AWS logs.

⑥ Inspector

- Automated vulnerability assessment service.
- Scans EC2 instances, containers, Lambda.
- Helps improve security and compliance.

How They Work Together

REMEMBER

- Security is a shared responsibility.
- Enable logging and monitoring everywhere.
- Principle of Least Privilege always.

BEST PRACTICE

- Enable MFA for all users.
- Rotate secrets and keys regularly.
- Use IAM roles instead of long-term keys.

PRO TIP

Combine GuardDuty, Security Hub and CloudWatch for complete visibility!

USE CASES

- Protect web apps with WAF + Shield.
- Secure data using KMS and Secrets Manager.
- Detect threats early with GuardDuty.

@qa_insights

DevOps Architecture

AWS

A complete end-to-end DevOps architecture for modern cloud-native applications.

1 GitHub
- Source code repository.
- Branching & Pull Requests.
- Webhooks trigger CI/CD pipeline.

2 Jenkins
- CI/CD automation server.
- Build, Test and Package code.
- Orchestrates the entire pipeline.

3 Docker
- Containerize applications.
- Consistent environments.
- Lightweight & portable.

4 ECR
- Elastic Container Registry.
- Store and manage Docker images.
- Secure, scalable image storage.

5 EKS / EC2
- Run containers on EKS (Kubernetes) or EC2 instances.
- Auto-scaling, high availability.
- Manage workloads efficiently.

6 Monitoring
- Monitor logs, metrics, and alerts.
- CloudWatch, Prometheus, Grafana.
- Ensure system health & uptime.

End-to-End DevOps Flow

1. **Code Commit** — Developer: Push code to GitHub →
2. **Build & Test** — Jenkins: Build code, run unit tests →
3. **Build Image** — Jenkins: Create Docker image →
4. **Push Image to ECR** — Jenkins: Push image to ECR repository →
5. **Deploy to EKS / EC2** — Jenkins: Deploy image to EKS cluster or EC2 instance →
6. **Monitor & Alert** — CloudWatch / Monitoring: Monitor application & get alerts

Feedback Loop

Architecture Overview

Developers → GitHub → Jenkins → ECR → EKS / EC2 (Pods / Containers) → Monitoring (CloudWatch Dashboards & Alerts)

REMEMBER
- Automate everything.
- Small, frequent deployments.
- Monitor continuously.

BEST PRACTICE
- Keep pipeline stages small and independent.
- Use Infrastructure as Code (IaC).
- Implement security scanning in pipeline.

PRO TIP
Use Docker + ECR + EKS for scalable and reliable containerized deployments.

KEY TAKEAWAY
Automated CI/CD + Containers + Orchestration + Monitoring = Reliable, Scalable and Secure Applications!

@qa_insights

AWS CLI & Essential Commands

Manage and automate AWS services from your terminal using the AWS Command Line Interface.

1 Configure CLI

- Install AWS CLI.
- Run configure to set credentials.
- Store access keys securely.

```
aws configure
AWS Access Key ID     [****************]
AWS Secret Access Key [****************]
Default region name   [ap-south-1]
Default output format [json]
```

2 S3 Commands

- List buckets
- Upload / Download files
- Sync files or folders
- Remove objects

```
aws s3 ls                                   # List buckets
aws s3 cp file.txt s3://my-bucket/          # Upload
aws s3 cp s3://my-bucket/file.txt .         # Download
aws s3 sync ./data s3://my-bucket/data      # Sync
aws s3 rm s3://my-bucket/file.txt           # Delete
```

3 EC2 Commands

- Manage EC2 instances
- Start, Stop, Reboot
- Describe instances
- Create key pair

```
aws ec2 describe-instances
aws ec2 start-instances --instance-ids i-1234567890
aws ec2 stop-instances --instance-ids i-1234567890
aws ec2 reboot-instances --instance-ids i-1234567890
aws ec2 create-key-pair --key-name my-key
```

4 IAM Commands

- Create users and groups
- Attach / List policies
- Manage access keys

```
aws iam list-users
aws iam create-user --user-name newuser
aws iam list-groups
aws iam attach-user-policy \
--user-name newuser --policy-arn arn:aws:iam::aws:policy/ReadOnlyAccess
```

5 CloudFormation Commands

- Create, list and delete stacks
- Validate templates
- Describe stack events

```
aws cloudformation create-stack \
  --stack-name my-stack --template-body file://template.yaml
aws cloudformation list-stacks
aws cloudformation describe-stacks --stack-name my-stack
aws cloudformation delete-stack --stack-name my-stack
```

6 Useful Shortcuts

- Save time with these handy CLI options.

```
--profile <name>       # Use specific profile
--region <region>      # Set region
--output table         # Human readable output
--query "<expr>"       # Filter output (JMESPath)
--no-paginate          # Disable pagination
--debug                # Detailed debug logs
```

REMEMBER

- Keep your Access Keys secure.
- Use IAM roles instead of long-term keys.
- Always set region and output format.

BEST PRACTICE

- Use least privilege IAM policies.
- Use --profile for multiple accounts.
- Automate with scripts + CLI.

PRO TIP

Use aws help <command> to get help and examples for any AWS command!

USE CASES

- Automate infrastructure provisioning
- Backup, deploy, monitor and manage AWS
- Integrate CLI in CI/CD pipelines

@qa_insights

17

AWS Best Practices

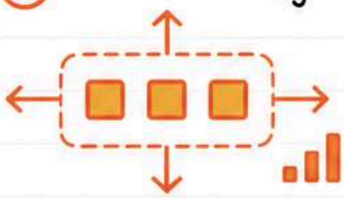
Follow these best practices to build secure, reliable, scalable and cost-effective cloud solutions.

1 Least Privilege IAM

- Grant only the minimum permissions required to perform a task.
- Use IAM roles for AWS services instead of access keys.
- Regularly review and remove unused permissions.

```
# Example: IAM Policy (Deny by default)
{
 "Version": "2012-10-17",
 "Statement": [{ "Effect": "Allow",
 "Action": "s3:ListBucket",
 "Resource": "arn:aws:s3:::my-bucket" }]
}
```

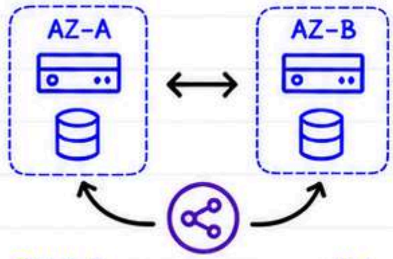
2 Use Auto Scaling



- Automatically add/remove resources based on demand.
- Maintain application availability and performance.
- Optimize costs by scaling in and out.

Use CloudWatch metrics (CPU, Memory, Requests) to trigger scaling actions.

3 Multi-AZ Deployment



- Distribute resources across multiple Availability Zones.
- Improves fault tolerance and high availability.
- Protects against AZ failures.

Use ALB, RDS Multi-AZ, EFS, Route 53 for HA.

4 Enable Monitoring

- Monitor resources, applications and user activity.
- Set up CloudWatch Alarms for critical metrics.
- Centralize logs for analysis and troubleshooting.

Use: CloudWatch, CloudTrail, X-Ray, AWS Config.

5 Infrastructure as Code

- Use CloudFormation, Terraform or CDK.
- Version control your infrastructure.
- Enables repeatability, consistency and quick recovery.

```
# CloudFormation Example
Resources:
  MyBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: my-bucket
```

6 Cost Optimization Tips

- Right-size your instances.
- Use Reserved Instances or Savings Plans.
- Stop unused resources.
- Use S3 Lifecycle policies.
- Monitor costs with AWS Cost Explorer.

Pay for what you use. Optimize continuously!

REMEMBER

- Security, Reliability and Cost go hand-in-hand.
- Automate, Monitor and Optimize continuously.
- Follow Well-Architected Framework.

BEST PRACTICE

- Design for failure, not for success.
- Automate everything you can.
- Review and improve regularly.

PRO TIP

Combine Automation (IaC) + Monitoring + Auto Scaling for Secure, Scalable and Cost-Effective AWS solutions!

USE CASES

- High Availability Web Applications
- Scalable Microservices
- Disaster Recovery Solutions

@qa_insights

18

Ultimate AWS Cheat Sheet

aws

One-page quick revision of everything you need to know!

1 Most Used Services

EC2	Virtual Servers in the cloud
S3	Scalable Object Storage
RDS	Managed Relational Database
Lambda	Serverless Compute
VPC	Isolated Virtual Network
IAM	Access Management
CloudWatch	Monitoring & Logs
Route 53	DNS & Domain Management
ELB	Load Balancing
CloudFront	CDN & Content Delivery

2 Service Categories

Compute	EC2, Lambda, ECS, EKS
Storage	S3, EBS, EFS, Glacier
Database	RDS, DynamoDB, Aurora
Networking	VPC, ELB, Route 53, CloudFront
Security	IAM, KMS, Security Group, WAF
Management	CloudWatch, CloudTrail, Config
DevOps	CodePipeline, CodeBuild, CodeDeploy

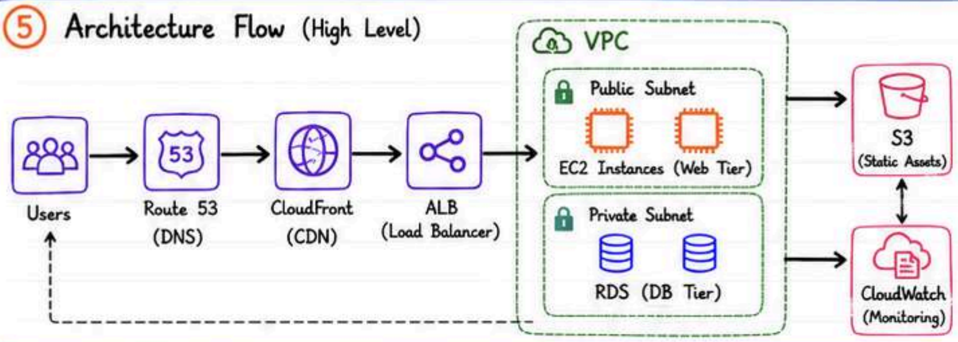
3 Interview Revision

- AWS is a secure, scalable and highly available cloud platform.
- Pay-as-you-go pricing model.
- Global Infrastructure with Regions & AZs.
- IAM for access management.
- S3 for storage, EC2 for compute.
- RDS for managed databases.
- VPC for network isolation.
- CloudWatch for monitoring.
- Infrastructure as Code with CloudFormation / CDK.
- High Availability with Multi-AZ & Auto Scaling.

4 Common Ports

22	SSH
80	HTTP
443	HTTPS
21	FTP
25	SMTP
53	DNS
3389	RDP
3306	MySQL
5432	PostgreSQL
1521	Oracle DB

5 Architecture Flow (High Level)



6 Certification Roadmap

- Cloud Practitioner (Foundational)
- Solutions Architect Associate (Architect)
- DevOps Engineer Professional (Operations)
- Solutions Architect Professional (Advanced)
- Specialty Certifications (Security / DevOps / Data / ML)

7 Top 20 Interview Topics

1. AWS Global Infrastructure
2. Regions vs AZs
3. IAM Users vs Roles
4. VPC Components
5. Security Groups vs NACL
6. EC2 Instance Types
7. EBS vs Instance Store
8. S3 Storage Classes
9. ELB Types
10. Auto Scaling
11. RDS Multi-AZ
12. Route 53 Routing Policies
13. CloudFront vs S3
14. AWS Lambda Basics
15. CloudWatch Metrics & Alarms
16. CloudFormation Basics
17. Cost Optimization
18. Shared Responsibility Model
19. Well-Architected Framework
20. Disaster Recovery

8 One-page Quick Revision

- ☑ Compute: EC2, Lambda, ECS, EKS
- ☑ Storage: S3, EBS, EFS, Glacier
- ☑ Database: RDS, DynamoDB, Aurora
- ☑ Networking: VPC, ELB, Route 53
- ☑ Security: IAM, KMS, SG, WAF
- ☑ Monitoring: CloudWatch, CloudTrail
- ☑ DevOps: CodePipeline, CodeBuild
- ☑ IaC: CloudFormation, CDK
- ☑ High Availability: Multi-AZ, ASG
- ☑ Cost: Right-sizing, Reserved Instances, S3 Lifecycle

REMEMBER

- Always follow the Principle of Least Privilege.
- Monitor everything and set alarms.
- Design for failure, not for success.

BEST PRACTICE

- Use Multi-AZ for critical resources.
- Automate with IaC and CI/CD.
- Tag your resources and control costs.

PRO TIP

- Use AWS Trusted Advisor.
- Enable IAM MFA.
- Backup, replicate & test DR plans.

IMPORTANT

Security + Monitoring + Automation = Reliable, Scalable & Cost-effective AWS Solutions!

FOCUS AREAS

- Security
- Performance
- Reliability
- Cost Optimization
- Operational Excellence

SUCCESS FORMULA

Right Architecture + Best Practices + Automation = Cloud Success